Ondos – A Maze Game

Glove-Controlled Labyrinth with an AI Solver

Project Proposal

Korzion (Group 2)
CSE 396 – Computer Engineering Project
Gebze Technical University

October 2026

GitHub: `github.com/korzion/ondos`

Team Members

Ahmet Arif Aygün
Beyzanur Bilir
Enis Talha Şahin
Süleyman Mevlid Uzun
Mohammed Z A Alaklouk
Talha Tanrıverdi
Mert Sezer Güler
İbrahim Ahmetoğlu
Yağmur Gül
Aayan Mustafa

Contents

1 Introduction **2**

2 Game Flow **3**

3 Modules **4**

4 Final Demonstration **8**

5 Team **9**

1 Introduction

Project name: Ondos – A Maze Game

Project scope: Ondos is a physical labyrinth game with a learning computer player. A **real maze with a ball** sits on a platform that can be tilted in two directions. The ball starts at the start point, and the aim is to roll it into the exit hole without getting stuck in dead ends. The maze has **at least one exit**.

The player does not use buttons or a joystick. The player wears a **glove**, and when they tilt their hand, the glove measures the angle and the platform tilts the maze in the same direction, in real time. The glove also vibrates to warn the player, for example when the ball moves away from the exit.

A camera above the maze follows the game with **image processing**. At the start it finds the walls, the start point and the exit. During the game it finds the ball in every frame, and it sees when the ball falls into the exit, which ends the game.

While the game is being played, the control station on the computer shows it in two ways. A **3D simulation** of the platform, the maze and the ball moves together with the real one, and **live 2D plots** show the path of the ball on the maze, the ball position and speed, the platform angle and the hand angle over time. From the same screen the game is **started, paused and stopped**, and an emergency stop brings the platform back to flat. **Control variables** such as the maximum tilt angle, the platform speed, the glove sensitivity and the vibration warnings can be changed while the game is running. Every game is **logged** with time stamps, so it can be replayed in 3D later.

The second part of the project is a **learning AI**. It first learns from the logged games of the human players, and then practises in the 3D simulation, where it can play many games quickly. After training, it plays the real maze by itself, and its time and success rate are compared with the players and with a simple controller that does not learn.

Aim of the project: To build a complete system in which hardware, image processing, simulation and machine learning work together, and to show that a computer can learn a physical skill, keeping a rolling ball under control, from human examples and from practice.

2 Game Flow

1. **Setup.** The maze is placed on the platform. The camera reads the walls, the start point and the exit, and the control station shows the maze map.
2. **Start.** On the control station, the user selects the mode (player or AI) and starts the game. The ball is placed at the start point.
3. **Playing.**
   - *Player mode:* the player tilts their hand, the glove sends the hand angle to the platform, and the maze tilts in the same direction. The glove vibrates when a warning rule is triggered.
   - *AI mode:* the camera gives the position of the ball, a path to the exit is calculated, and the AI decides how to tilt the maze.
4. **Monitoring.** During the game, the 3D model and the live graphs are updated and the settings can be changed. The game can be paused or stopped at any time, and the emergency stop brings the platform back to the flat position.
5. **End.** When the camera sees the ball fall into the exit, the game ends. The time and the full game log are saved.
6. **Learning.** The saved player games are used to train the AI, first as examples and then together with practice in the simulation. The trained AI then plays in AI mode, and its results are compared with the players and the simple controller.

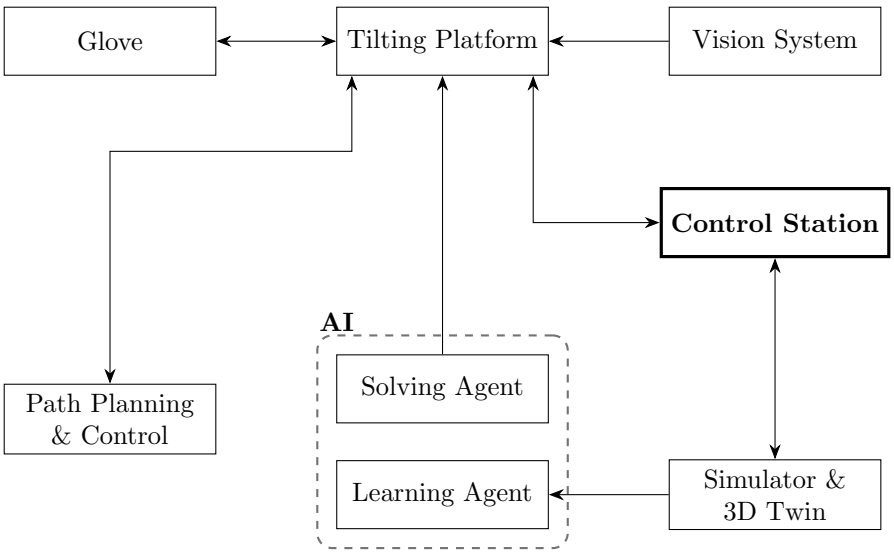


Figure 1: System overview: how the seven modules are connected.

3 Modules

Figure 1 shows how the modules are connected. Each module will be developed and tested separately, so that it can be shown on its own on the module demo day. The images below are concept renders of the finished system. They show what each module does, not how it will be built; the hardware is chosen in the module design documents.

A) Tilting Platform

- A maze that can be tilted in two directions (Figure 2).
- At least one exit hole. The ball is collected after it falls through.
- Tilts only within safe limits.
- *Demo:* the platform tilts to a requested angle.

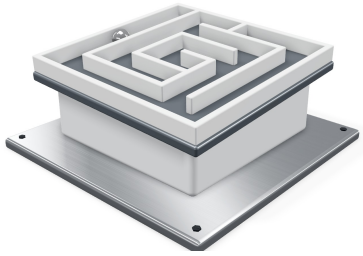


Figure 2: Tilting platform: the maze board sits on a drive unit that tilts it on two axes.

B) Glove

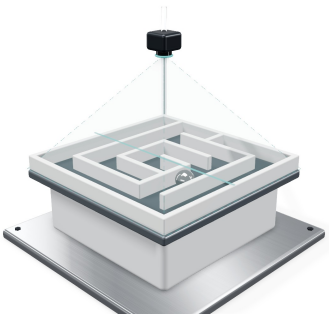
- Measures the tilt of the player's hand (Figure 3).
- Sends this data to the platform without cables.
- Vibrates to warn the player, for example:
  - when the ball has not moved for a long time,
  - when the ball moves away from the exit,
  - when the player is slower than the best previous run.
- Works on battery.
- *Demo:* the hand angle is shown on the screen and the vibration patterns are tested.



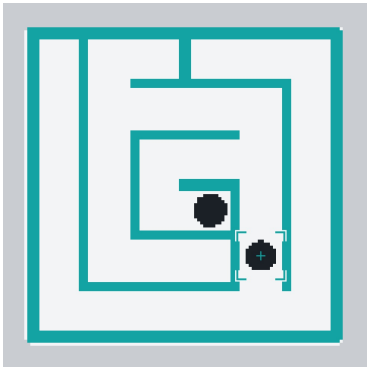
Figure 3: Glove: the electronics sit on the back of the hand; it works without cables.

C) Vision System

- A camera that sees the whole maze (Figure 4).
- Finds the walls, the start point and the exit in the image.
- Finds the position and speed of the ball in every frame.
- Detects when the ball falls into the exit.
- *Demo:* the maze and the ball are marked on the live camera image.



(a) The camera above the board scans the maze.



(b) The camera's view, with the ball tracked.

Figure 4: Vision system: the camera finds the maze and tracks the ball.

D) Simulator & 3D Twin

- A computer model of the platform, the maze and the ball (Figure 5).
- It should behave as close as possible to the real system.
- It will be used to:
  - train the AI,
  - test the software before the hardware is ready,
  - show the game in 3D while it is being played.
- *Demo:* the virtual maze can be played with the keyboard.

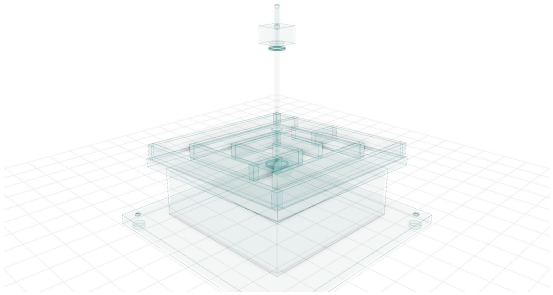


Figure 5: Simulator & 3D twin: a computer model of the platform, the maze, the camera and the ball.

E) Path Planning & Control

- Calculates a path from the ball to the exit (Figure 6).
- Keeps track of how far the ball has moved along this path. The AI and the glove warnings use this information.
- Moves the platform to the requested angle in every mode.
- Includes a simple controller that does not learn. It is used for comparison with the AI.
- *Demo:* the simple controller solves the maze in the simulator.

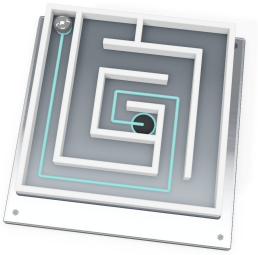


Figure 6: Path planning: the path from the start point to the exit hole, calculated on the maze.

F) AI

- *Learning Agent:* learns how to move the ball to the exit (Figure 7),
  - first from the games played by people,
  - then by practising in the simulator.
- *Solving Agent:* the trained AI, which plays on the real platform.
- Its results are compared with the human players and the simple controller.
- *Demo:* the AI learns and solves the maze in the simulator.

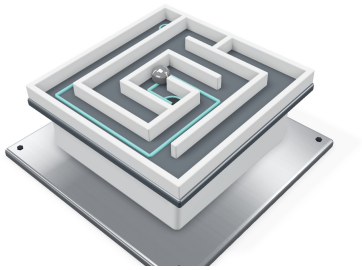


Figure 7: AI: the solving agent tilts the platform by itself and moves the ball along the path.

G) Control Station

- A program on the computer that starts and stops the game, with an emergency stop (Figure 8).
- Switches between player mode and AI mode.
- Shows live 2D graphs and the 3D model of the game.
- Settings can be changed while the game is running.
- Checks the game rules and saves every game. The saved games are used to train the AI.
- *Demo:* the program runs with data coming from the simulator.

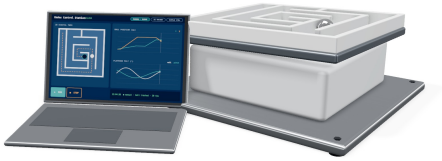


Figure 8: Control station: the 3D digital twin and live plots on the computer, next to the platform.

4 Final Demonstration

- A player solves the maze using the glove.
- The AI solves the same maze on the real platform, and its results are compared with the human players and the simple controller.
- The 3D model and the live graphs work during all games.

5 Team

The team has ten members. Each module has two to four members, depending on its workload, and each member works in two or three modules. This way, every module can continue even if some members are not available.

Member	Tilting Platform	Glove	Vision System	Simulator & 3D Twin	Path Planning & Control	AI	Control Station
Ahmet Arif Aygün		•					•
Beyzanur Bilir	•						•
Enis Talha Şahin	•			•			
Süleyman Mevlid Uzun					•	•	
Mohammed Z A Alaklouk	•			•			•
Talha Tanrıverdi			•			•	
Mert Sezer Güler			•			•	
İbrahim Ahmetoğlu			•		•	•	
Yağmur Gül				•	•		
Aayan Mustafa	•	•					

The source code, documents and the contribution history of every member are kept in the project's GitHub repository: `github.com/korzion/ondos`. Each member's contributions per module (commits, changed lines and reviews) can be followed on the team activity page: `korzion.github.io/ondos`